Reviewer Pack — Plethysm Coefficient Gap in Monotone Circuit Complexity for …

Entry a40f78da41e6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 15:49:10 UTC

Plethysm Coefficient Gap in Monotone Circuit Complexity for Permanent vs Determinant

Entry ID: `a40f78da41e6` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-11 15:49:10 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Monotone Boolean Circuit Size

Statement:

> For random 3-SAT instances with n variables, the plethysm coefficient of the permanent polynomial in the symmetric power S^k(V) exceeds that of the determinant polynomial by Ω(n^{1.5}) for k = ⌈n^{1.5}⌉, and this gap persists under linear substitutions preserving the monomial basis.

Rationale (proposer's reasoning):

> Schur-Weyl duality decomposes tensor powers into irreducible representations, while monotone circuit complexity measures algebraic complexity. Plethysm coefficients encode how symmetric functions decompose, offering a bridge between representation-theoretic structure and circuit lower bounds.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `be8222c7c0701bd5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM_1	LLM_2
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from collections import Counter

def factorial(n):
    if n == 0:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def hook_length_formula(n, k):
    numerator = 1
    denominator = (factorial(k) ** 2) * factorial(n - k)
    for i in range(1, n + 1):
        for j in range(1, k + 1):
            numerator *= (i + j - 1)
    return numerator / denominator

def plethysm_coefficient(n, k):
    if k == 0:
        return 1
    permanent_coeff = hook_length_formula(n, k) / hook_length_formula(n, 1)
    determinant_coeff = hook_length_formula(n, k - 1) / hook_length_formula(n, 1)
    return permanent_coeff - determinant_coeff

def generate_3sat_instance(n):
    clauses = []
    for _ in range(2 * n):
        clause = random.sample(range(1, n + 1), 3)
        if random.choice([True, False]):
            clause = [-x for x in clause]
        clauses.append(clause)
    return clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        k = math.ceil(n ** 1.5)
        instance = generate_3sat_instance(n)
        permanent_coeff = plethysm_coefficient(n, k)
        determinant_coeff = plethysm_coefficient(n, k - 1)
        gap = permanent_coeff - determinant_coeff
        if gap < n ** 1.5:
            conjecture_holds = False
            counterexample = f"n={n}, k={k}, gap={gap}"
            break
        total_metric_value += gap
        instances_tested += 1

    return {
        "metric_name": "plethysm_coeff_gap",
        "metric_value": total_metric_value / instances_tested if instances_tested > 0 else 0,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96db9ebb.py", line 84, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96db9ebb.py", line 60, in run_trial
    permanent_coeff = plethysm_coefficient(n, k)
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96db9ebb.py", line 36, in plethysm_coef
    permanent_coeff = hook_length_formula(n, k) / hook_length_formula(n, 1)
                      ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96db9ebb.py", line 31, in hook_length_f
    return numerator / denominator
           ~~~~~~~~~~^~~~~~~~~~~~~
OverflowError: integer division result too large for a float
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with overflow error, preventing evaluation of plethysm coefficients. | next: Implement arbitrary-precision arithmetic or parameter constraints to avoid numerical overflow

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109928
2	propose	ollama_remote	qwen3:8b	0	0	32807
3	preregistration	ollama_remote	qwen3:8b	0	0	24139
4	novelty	ollama_remote	qwen3:8b	0	0	20640
5	novelty	ollama_remote	qwen3:8b	0	0	14158
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39603
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10398
8	judge	ollama_remote	qwen3:8b	0	0	16903

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 268576 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/a40f78da41e6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a40f78da41e6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a40f78da41e6.tar.gz` (if generated)